МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

КУРСОВАЯ РАБОТА

по дисциплине «Введение в нереляционные базы данных»

Тема: Приложение для анализа пешеходных дорожек в СПб

Студент гр. 2300 _____ Голов О.С.

Преподаватель _____ Заславский М.М.

Санкт-Петербург
2026

ЗАДАНИЕ
НА КУРСОВУЮ РАБОТУ

Студент Голов О.С.

Группа 2300

Тема работы: приложение реализующее импорт данных из openstreet maps в neo4j для анализа пешеходных дорожек в СПб

Исходные данные:

Необходимо пройти курс на moevm.info на максимальный балл, подготовить приложение реализующее импорт данных из openstreet maps в neo4j для анализа пешеходных дорожек в СПб.

Содержание пояснительной записки:

Введение, Сценарии использования, Модель данных, Разработанное приложение, Выводы, Список литературы, Приложения.

Предполагаемый объем пояснительной записки:

Не менее 15 страниц.

Дата выдачи задания: 03.03.2026

Дата сдачи реферата: 23.04.2026

Дата защиты реферата: 23.04.2026

Студент _____ Голов О.С.

Преподаватель _____ Заславский М.М.

АННОТАЦИЯ

Проект посвящён проектированию и разработке приложения «Анализ путей», сфокусированном на построении маршрутов и анализе их опасности. Разработка велась с использованием языка программирования Python. В ходе разработки был создан прототип приложения включающий такие системы как: отображение карты и узлов сети дорожек на ней, построение пешеходных маршрутов из точки в точку и анализ маршрутов на предмет близости к проезжей части. Имеется множество путей для дальнейшего улучшения приложения, вроде улучшения пользовательского опыта, и увеличения количества функций.

SUMMARY

The project is dedicated to the design and development of the Path Analysis application, focused on route construction and hazard analysis. The development was carried out using the Python programming language. During the development, a prototype application was created that includes such systems as: displaying a map and nodes of a network of paths on it, building walking routes from point to point and analyzing routes for proximity to the roadway. There are many ways to further improve the application, such as improving the user experience and increasing the number of features.

СОДЕРЖАНИЕ

	Введение	5
1.	Сценарии использования	6
2.	Модель данных	7
2.1.	Нереляционная модель данных	7
2.2.	Аналог модели данных для SQL СУБД	10
2.3	Сравнение	12
3.	Разработанное приложение	13
3.1.	Схема архитектуры	13
3.2.	Использованные технологии (внешние)	13
3.3.	Использованные модули/системные библиотеки	13
4.	Выводы	14
4.1.	Достигнутые результаты	14
4.2.	Недостатки и пути для улучшения полученного решения	14
4.3.	Будущее развитие решения	14
	Заключение	14
	Список использованных источников	15
	Приложение А. Инструкция для пользователя	16
	Приложение Б. Снимки экрана приложения	16

ВВЕДЕНИЕ

А. Актульность проблемы

В современном мире для построения удачных маршрутов необходимо понимать множество аспектов, вроде самой возможности дойти до желаемой точки из той точки где находится человек, и то насколько этот путь будет опасным. И только получив удоволетворительный ответ можно отправляться в путь, но даже так, неизветсно как построить маршрут.

В. Постановка задачи

создайте приложение, которое реализует импорт данных из openstreet maps в neo4j для анализа пешеходных дорожек в СПб позволяющее построить путь из точки а в точку б, её возможность, а также то насколько опасен путь, в контексте близости к проезжим частям.

С. Предлагаемое решение

В качетсве решения проблемы было разработано приложение «Анализ пути», которое включает в себя следующие элементы:

1. Построение маршрута из точки А в точку Б
2. Анализ маршрута на близость к проезжим частям
3. Просмотр существующих точек на карте

1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

1.1.1 Сценарий использования «как пользователь загружает данные в программу массово»

Действующее лицо: администратор

Основной сценарий:

1) пользователь запускает скрипт import_data.py

2) система обращается к api openstreetmap

3) алгоритм вычисляет расстояние до проезжих частей

4) данные собираются в списки словарей и массово загружаются в Neo4j

1.1.2 Сценарий использования «как пользователь загружает данные в программу вручную»

1) пользователь может добавить при запуске скрипта import_data.py желаемые флаги

1.2.1 Сценарий использования «как данные отображаются для пользователя и что он должен сделать для их отображения»

1) пользователь видит перед собой карту и поля с точкой А и точкой Б

2) пользователь может нажать кнопку «показать все узлы»

3) красные маркеры это те которые рядом с проезжими частями, а зелёные в безопасных зонах

4)пользователь может тыкнуть по экрану и слева появится предложение сделать ближайщую точку точкой А или точкой Б

5) пользователь выбирает желаемые точки и видит на карте путь с пометками об опасных зонах и безопасных и примерный индекс опасности

1.3.1 Сценарий использования «какие действия должен сделать пользователь для проведения анализа данных»

1) пользователь строит маршрут

2) пользователь видит опасность и безопасноть пути

3) система показывает индекс опасности

4) пользователь выбирает желаемые точки и видит на карте путь с пометками об опасных зонах и безопасных и примерный индекс опасности

5) пользователь может нажать кнопку «показать все узлы»

1.4.1 Сценарий использования «как пользователю получить копию хранимых в программе данных в машиночитаемом формате»

1) пользователь видит перед собой карту и поля с точкой А и точкой Б

2) пользователь собирает путь

3) экспортирует в желаемом виде путь в виде списка узлов

1.5 Вывод о том, какие операции будут преобладать для решения

Операции записи выполняются разово, при массовом импорте данных через api

Операции чтения происходят постоянно: при запуске, при нажатии на карту, при построении маршрута, при включении всех точек

2. МОДЕЛЬ ДАННЫХ

2.1.1 Графичексое представление

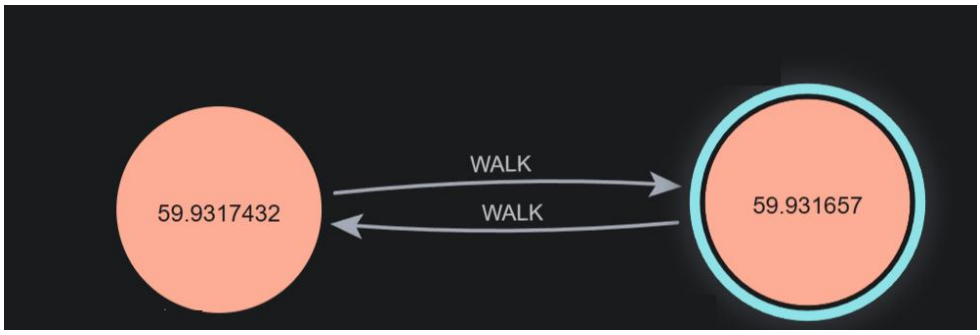

Рисунок 1 – представление связей в графе

2.1.2. Описание назначений коллекций, типов данных и сущностей

Коллекция узлов Intersection:

Osmid (Integer, 8 байт) id узла

Lat (Float, 8 Байт) широта

Lon (Float, 8 Байт) долгота

Near_road (Boolean, 1 байт) флаг опасности

Метаданные ~15 байт

Вес узла ~40 байт

Пример данных Intersection:

```
{
  "osmid": 253160451,
  "lat": 59.9343,
  "lon": 30.3246,
  "near_road": false
}
```

Коллекция узлов Walk:

Length (Float, 8 Байт) длина перехода в метрах

Метаданные ~34 байта

Вес узла ~42 байта

Пример данных Walk:

```
{
  "length": 15.4
}
```

2.1.3. Оценка удельного объема информации, хранимой в модели

Пусть N – количество узлов

K – среднее количество исходящих рёбер

Тогда V_node = N*40

V_link = N*k*42 = ~105N

Индексы и служебные данные ~500кбайт

Итоговая формула объёма = V = 40N+105N +500кбайт = 145N + 500кбайт

Следовательно характер роста: линейный

2.1.4. Запросы к модели, с помощью которых реализуются сценарии использования

2.4.1 Построение кратчайшего маршрута

```
MATCH p=shortestPath(
    (start:Intersection {osmid: $start_id})-[:WALK*]-(end:Intersection {osmid: $end_id})
)
UNWIND nodes(p) AS n
RETURN n.osmid AS osmid, n.lat AS lat, n.lon AS lon, n.near_road AS near_road
```

2.4.2. поиск ближайшего узла к точке

```
MATCH (n:Intersection)
WITH n, (n.lat - $lat)^2 + (n.lon - $lon)^2 AS dist
ORDER BY dist ASC
LIMIT 1
RETURN n.osmid AS osmid
```

2.4.3. анализ безопасности маршрута

```
MATCH p=shortestPath((start:Intersection {osmid: $id1})-[:WALK*]-(end:Intersection {osmid: $id2}))
UNWIND nodes(p) AS node
RETURN count(node) AS total_nodes,
```

sum(CASE WHEN node.near_road = true THEN 1 ELSE 0 END) AS dangerous_nodes

2.2. Аналог модели данных для SQL СУБД

2.2.2. Описание назначений коллекций, типов данных и сущностей

Таблица Intersection:

Id (serial pk, 4 байт) суррогатный ключ таблицы

Osmid (bigint, 8 байт) id узла

Lat (double, 8 Байт) широта

Lon (double, 8 Байт) долгота

Near_road (Boolean, 1 байт) флаг опасности

Заголовок строки ~24 байт

Вес узла ~53 байт

Таблица Walk_edges:

Id (serial pk, 4 байт) суррогатный ключ таблицы

Start_id (int, 4 Байт) ссылка на intersections.id

end_id (int, 4 Байт) ссылка на intersections.id

length (double, 4 Байт) длина в метрах

Заголовок строки ~24 байт

Вес узла ~44 байта

2.2.3. Оценка удельного объема информации, хранимой в модели

Пусть N – количество узлов

K – среднее количество исходящих рёбер

Тогда V_node = N*53

V_link = N*k*42 = ~110N

Индексы и служебные данные ~1Мбайт

Итоговая формула объёма = V = 43N+110N +500кбайт = 63N + 1Мбайт

Следовательно характер роста: линейный

2.2.4. Запросы к модели, с помощью которых реализуются сценарии использования

2.2.4.1 Построение кратчайшего маршрута

```
WITH RECURSIVE route AS (
    SELECT id, lat, lon, near_road, 0::float as total_length
    FROM intersections WHERE osmid = {start_id}

    UNION ALL

    SELECT i.id, i.lat, i.lon, i.near_road, r.total_length + e.length
    FROM intersections i
    JOIN walk_edges e ON i.id = e.end_id
    JOIN route r ON e.start_id = r.id
    WHERE r.total_length < 5000
)
SELECT * FROM route
WHERE osmid = {end_id}
ORDER BY total_length ASC LIMIT 1;
```

2.2.4.2. поиск ближайшего узла к точке

```
MATCH (n:Intersection)
WITH n, (n.lat - $lat)^2 + (n.lon - $lon)^2 AS dist
ORDER BY dist ASC
LIMIT 1
RETURN n.osmid AS osmid
```

2.2.4.3. Поиск ближайшего узла

```
SELECT id, osmid
FROM intersections
ORDER BY (lat - {user_lat})^2 + (lon - {user_lon})^2 ASC
LIMIT 1;
```

2.3.1 Удельный объем информации - где данные занимают больший объем при прочих равных\

При прочих равных, sql модель будет занимать сильно больше памяти из-за того растёт быстрее в размерах с повышением количества узлов, и константно более тяжёлая за счёт индексов.

2.3.2 Количество запросов для совершения юзкейсов в зависимости от числа объектов в БД и прочих параметров

В случае сценария «поиск кратчайшего пути» у реляционной БД будет 1 запрос который будет рекурсивно обращаться к связанным узлам. Сложность будет O(LOG(E)) для одного только перехода на следующий узел, в то время как в нереляционной БД сложность будет O(E+ VlogV) по алгоритмы дейкстры для всего поиска.

2.3.3 Количество задействованных коллекций

В neo4j задействована 1 коллекция и 1 тип связи. В SQL используется 2 таблицы

2.3.4 Сложность запроса

Работа с neo4j проста из-за декларативного подхода, в то время как язык sql очень сложен для комплексных запросов.

2.4 Вывод

В случае работы с построением маршрутов гораздо лучше работать с графовой моделью, так как она идеально подходит к задаче ввиду того что построение маршрутов в целом строится на логике графов. Также она

производительнее и эффективнее по ресурсам. SQL может подойти для менее связных точек.

3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

3.1. Краткое описание

Приложение состоит из двух модулей: import_data.py и app.py, которые занимаются массовым импортом и логикой приложения соответственно.

Модуль с импортом загружает данные через api, парсит их и загружает в базу данных neo4j

Модуль с логикой занимается работой с картой и обращениями в базу данных

3.2. Использванные технологии

В качестве языка разработки был выбран python, так как является наиболее удобным для разработки. Основу базы данных была Neo4j за счёт своей графовости. Библиотека osmnx для обработки openstreetmap. Scipy для быстрого счёта расстояний узлов. Folium для вызова карты

3.3. Снимки экрана приложения

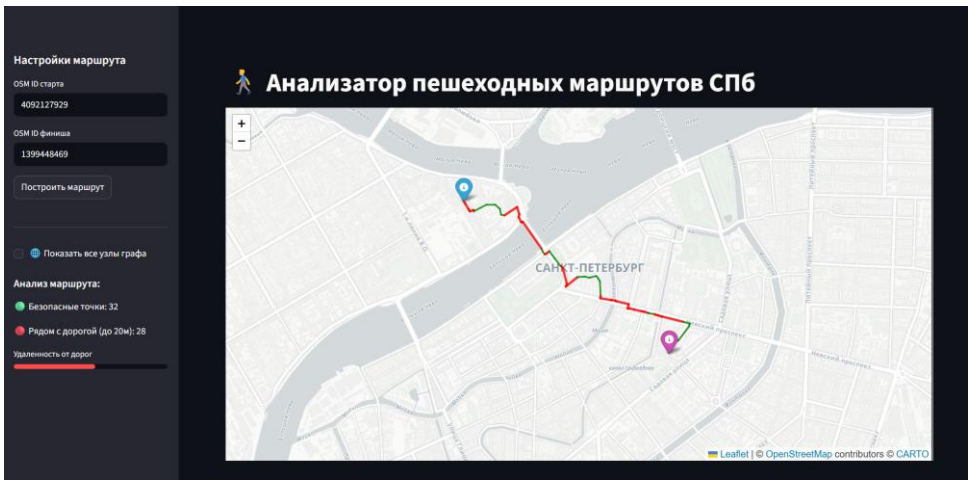

Рисунок 2 – изображение интерфейса приложения

4. ВЫВОДЫ

4.1. Достигнутые результаты

В ходе разработки приложения были освоены методики разработки приложений с поддержкой баз данных отличных от sql. Были получены навыки использования api, и работа с картами.

4.2. Недостатки и пути для улучшения полученного решения

Не самый оптимизированный алгоритм поиска

Малое количество настроек анализа

Отсутствие кэширования карты.

4.3. Будущее развитие решения

Переход на эвристические методы поиска пути

Интеграция данных об освещении, пешеходных переходах, светофорах, для более точного анализа

Кеширование карты

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было написано приложение для анализа маршрутов и их построения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. инструмент для интеграции Neo4j// Наименование сайта. URL:://https://neo4j.com/blog/neo4j-doc-manager-polyglot-persistence-mongodb/ (дата обращения: 23.04.2026).

2. репозиторий с кодом// Наименование сайта. URL:://https://github.com/avariiinaa/nosqls8

ПРИЛОЖЕНИЕ А

ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

1) docker run -d --name neo4j -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/password neo4j:latest

2) python3 -m venv venv

3) source venv/bin/activate

4) pip install -r requirements.txt

5) python import_data.py

6) streamlit run app.py

7) открыть браузер на порту http://localhost:8501 с логином и паролем "neo4j" "password" соответсвенно

ПРИЛОЖЕНИЕ Б

ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

Для построения маршурта тыкните на карту и выберете в меню её как точку финиши или старта, так же сделайте во второй раз, и вам откроется маршрут с пометками об опасных маршрутах. Так же можно нажать на кнопку показать все углы графа, чтобы увидеть все точки по которым осуществляется путь